

# 07a — Unit Testing

---

## Key Concepts

| Term                 | Definition                                                         |
|----------------------|--------------------------------------------------------------------|
| Unit Test            | Tests a single function/class in complete isolation                |
| Isolation            | No real DB, API, file system — all dependencies are faked          |
| Mock                 | Fake dependency that also verifies it was called correctly         |
| Stub                 | Fake dependency that just returns a fixed value                    |
| Fake                 | Simplified working implementation (e.g., in-memory DB)             |
| AAA Pattern          | Arrange → Act → Assert — structure of every unit test              |
| Deterministic        | Same test, same result every time — no flakiness                   |
| Dependency Injection | Pass dependencies in rather than creating them inside the function |

---

## How It Works

- 1. Identify the unit to test (one function or class)
  - 2. Replace all external dependencies (DB, API, time) with mocks/stubs
  - 3. Write test following AAA:
    - **Arrange** — set up inputs and configure mocks
    - **Act** — call the function
    - **Assert** — verify output or mock interactions
  - 4. Test all scenarios: happy path, edge cases, exception cases
- 

## Input Categories to Test

| Category         | Example ( <b>divide(a, b)</b> ) |
|------------------|---------------------------------|
| Happy path       | a=4, b=2 → result=2             |
| Zero numerator   | a=0, b=2 → result=0             |
| Division by zero | a=3, b=0 → throw exception      |
| Both zero        | a=0, b=0 → defined behavior?    |
| Negative numbers | a=-4, b=2 → result=-2           |

---

## Mock vs Stub vs Fake

| Type | Returns fake data | Verifies calls | Has real logic |
|------|-------------------|----------------|----------------|
|------|-------------------|----------------|----------------|

---

| Type | Returns fake data | Verifies calls | Has real logic   |
|------|-------------------|----------------|------------------|
| Stub | Yes               | No             | No               |
| Mock | Yes               | Yes            | No               |
| Fake | Yes               | No             | Yes (simplified) |

## Handling Uncontrollable Dependencies

**Problem:** `DateTime.Now`, random numbers, external APIs — can't control in tests.

**Solution:** Inject them as parameters instead of calling directly inside the function.

```
// Bad – untestable
public bool IsExpired(DateTime expiry) => DateTime.Now > expiry;

// Good – testable
public bool IsExpired(DateTime expiry, DateTime now) => now > expiry;
```

## Industry Examples

- **Amazon** — unit tests on discount/pricing logic catch calculation bugs before checkout breaks at scale
- **Google** — policy: untested code is treated as broken; unit tests catch regressions at commit time
- **Netflix** — streaming logic (bitrate selection, retry) is unit tested in isolation before integration

## Interview Questions

**Q1: What is unit testing?** Testing a single unit of logic in isolation with all external dependencies mocked, to verify correct behavior across all input scenarios.

**Q2: What's the difference between a mock and a stub?** A stub returns a fixed fake value. A mock does that plus verifies the dependency was called correctly (e.g., called once, called with specific args).

**Q3: Why mock external dependencies?** To keep tests fast, isolated, and deterministic. Real DB/API calls are slow, flaky, and hide which logic broke.

**Q4: What is the AAA pattern?** Arrange (set up inputs/mocks) → Act (call the function) → Assert (verify output/interactions).

**Q5: How do you test code that depends on `DateTime.Now`?** Inject time as a parameter. Never call `DateTime.Now` directly inside logic — inject it so tests can pass a controlled value.